

B-Trees – Comprehensive Learning Module

A deep-dive into one of the most critical data structures in computer science — the foundation of every relational database index, filesystem, and key-value store.

ADVANCED

DATA STRUCTURES

COMPUTER SCIENCE



Topic Classification

Field	Details
Topic Name	B-Trees
Category	Data Structures
Domain	Computer Science / Disk-Based / Multi-Way Balanced Trees
Topic Type	Conceptual + Algorithmic + System Design
Difficulty	Advanced
Prerequisites	Binary Trees, BST, AVL/Red-Black Trees, Basic I/O & Disk concepts
Industry Relevance	Critical — Foundation of every relational database index, filesystem, and key-value store

Recommended Learning Path



BST
Binary Search Trees



AVL / Red-Black
Self-balancing trees



B-Trees
You are here



B+ Trees → LSM Trees
Production variants

What Is a B-Tree?

Formal Definition

A B-Tree of **minimum degree t** is a rooted, balanced, multi-way search tree where:

- Every node has between **$t-1$** and **$2t-1$** keys (except root: at least 1 key)
- Every non-leaf node with k keys has exactly **$k+1$** children
- All leaves appear at the **same depth**
- Keys within each node are sorted in ascending order

Simple Explanation

Think of it as a wide, flat filing system: instead of a tall cabinet with single folders per shelf, you have a short cabinet where each shelf holds many folders, organized in order. Going to disk is like opening a filing cabinet drawer — expensive. B-Trees minimize the number of drawer opens (disk I/Os) by packing as much data as possible per shelf (disk page).

Core Purpose

Key Terminology

Term	Definition
Minimum Degree (t)	Minimum number of children for non-root internal nodes
Order (m)	Maximum number of children per node ($m = 2t$)
Node Capacity	Keys: $t-1$ to $2t-1$; Children: t to $2t$
Height	$O(\log_t n)$ — crucial for disk efficiency
Disk Page / Block	Unit of I/O; one B-Tree node = one disk page
Split	Divides an overfull node ($2t$ keys) into two nodes
Merge	Combines two underfull nodes during delete
Separator Key	Key pushed up to parent when a node splits
Median Key	Middle key of a full node — becomes the separator during split

i B-Tree Node (degree $t=3$, max 5 keys, max 6 children):

[K1][K2][K3][K4][K5] ← up to $2t-1 = 5$ keys

C0 C1 C2 C3 C4 C5 ← up to $2t = 6$ pointers

$K1 < K2 < K3 < K4 < K5$

Minimize Disk I/O

Each node sized to match one disk page (4KB–16KB)

$O(\log_t n)$ Access

Guaranteed disk accesses for any operation

Range Queries

Efficient prefix scans and sorted access

Why Does It Exist? The Disk vs. RAM Problem

100ns

RAM Access

Blazing fast in-memory reads

100 μ s

SSD Access

1,000 $\times$ slower than RAM

10ms

Disk Access

100,000 $\times$ slower than RAM

4

B-Tree Levels

For 1 billion records with $t \approx 250$

The Binary Tree Problem

A binary tree with 1 billion nodes has height ~ 30 . Each node access = 1 disk I/O = 30 disk reads for a lookup. At 10ms per disk access $\rightarrow$ **300ms per lookup**. Completely unacceptable for a database.

The B-Tree Solution

A B-Tree with page size 4KB, keys of 8 bytes $\rightarrow$ ~ 500 keys per node $\rightarrow t \approx 250$. Height $\approx \log_{250}(10^9) \approx$ **4 levels**. Same lookup needs **4 disk reads** $\rightarrow$ 40ms. That's the entire value proposition.

Business & Technical Drivers

Database Scalability

PostgreSQL, MySQL, Oracle all use B+ Tree indexes as default

Filesystems

NTFS, HFS+, ext4 use B-Tree variants for directory indexing

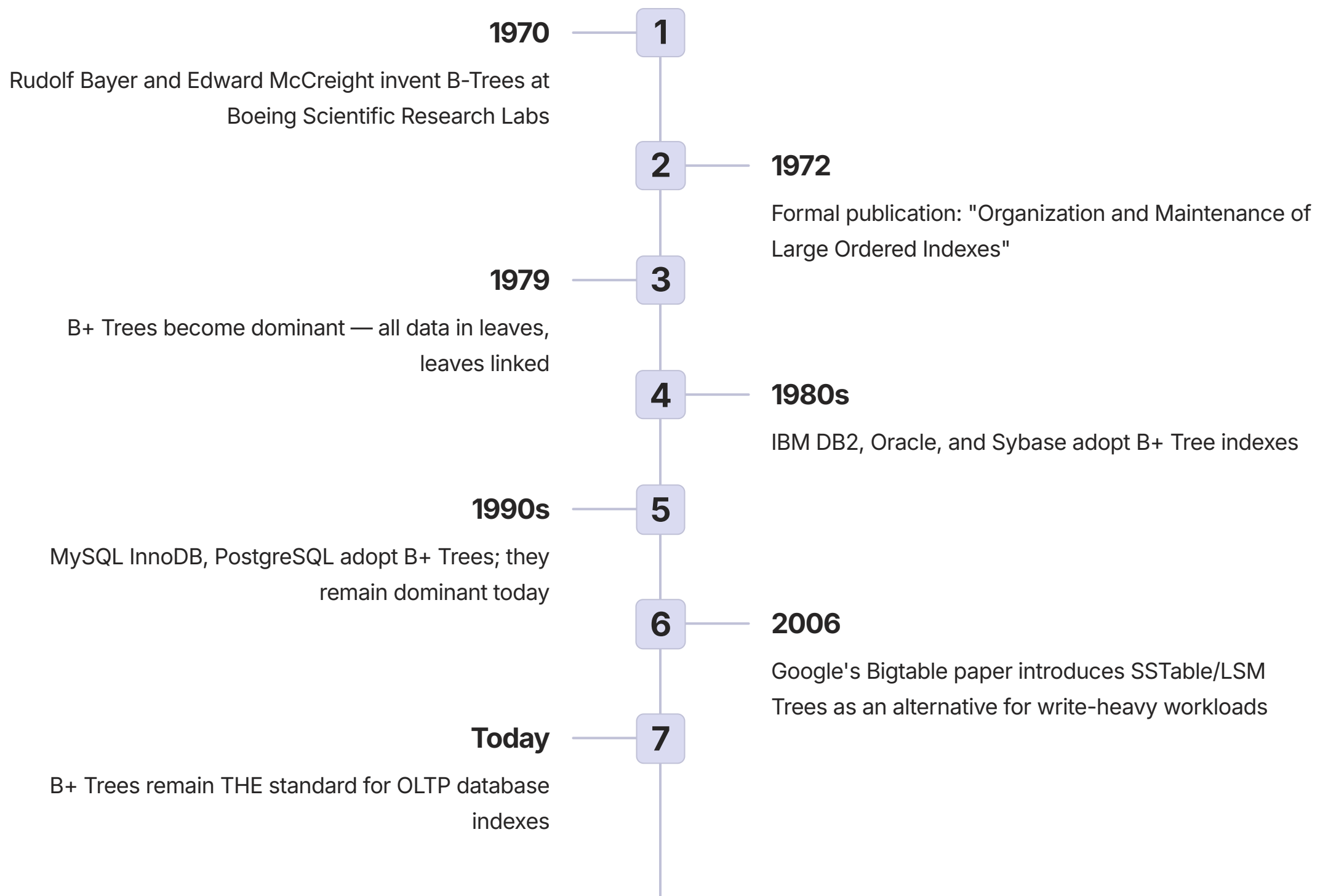
Key-Value Stores

LMDB, BerkeleyDB use B+ Trees as storage engine

Predictable Latency

Consistent $O(\log_t n)$ regardless of data distribution

Evolution & History

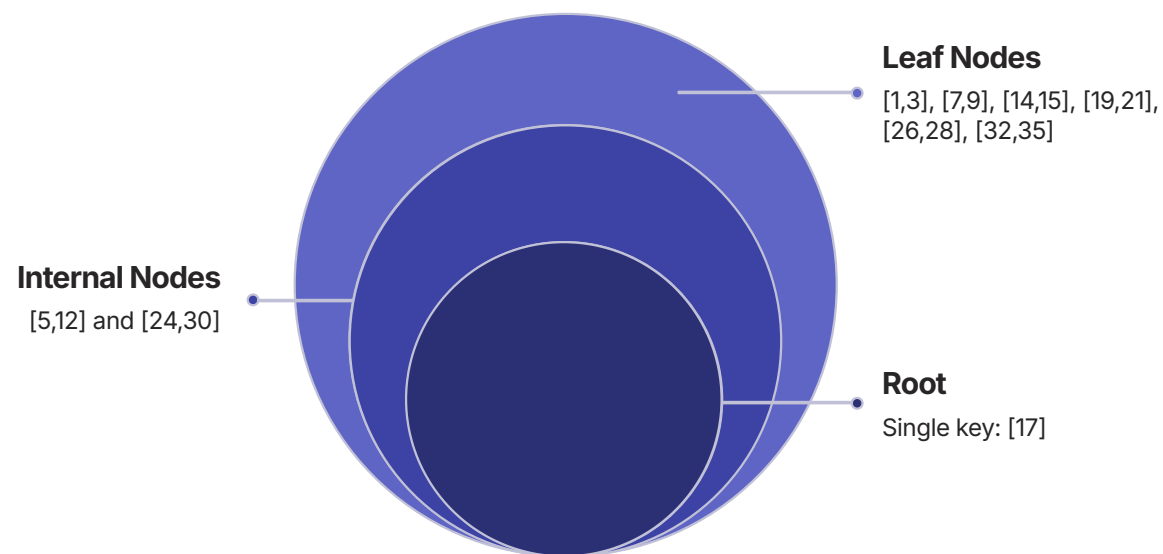


Previous Approaches & Their Limitations

Approach	Limitation
Sequential file scan	$O(n)$ lookup
Hash file organization	No range queries; overflow handling complex
Binary search on sorted file	$O(\log n)$ but poor for inserts (shifting data)
ISAM	Static index; doesn't handle inserts/deletes efficiently
In-memory BST as index	Doesn't survive restarts; doesn't scale past RAM

How It Works — Structure & Algorithms

B-Tree Structure (t=2, Order 4)



All leaves appear at the same depth, guaranteeing balanced worst-case performance for every operation.

Disk Page Alignment

① Disk Page = 4096 bytes | Key = 8 bytes | Pointer = 8 bytes
 $8n + 8(n+1) \leq 4096 \rightarrow n \leq 255$
 $\rightarrow t = 128 \rightarrow \text{height} \approx 3-4 \text{ levels for 1 billion records}$

Search Algorithm

```
FUNCTION search(node, key):  
    i = 1  
    WHILE i ≤ node.n AND key >  
node.key[i]:  
        i = i + 1  
  
    IF i ≤ node.n AND key ==  
node.key[i]:  
        RETURN (node, i) // Found  
    ELSE IF node.is_leaf:  
        RETURN NOT_FOUND  
    ELSE:  
        DISK_READ(node.child[i]) //  
Expensive!  
        RETURN search(node.child[i],  
key)
```

Cost: $O(\log_t n)$ disk reads
 $O(t \times \log_t n)$ comparisons

Disk I/O Count

Height $h = \lceil \log_t((n+1)/2) \rceil$

SEARCH: $h+1$ disk reads

INSERT: $h+1$ reads + $O(h)$ writes

DELETE: $h+1$ reads + $O(h)$ writes

$t=1000, n=10^9$: $h \approx 3 \rightarrow 4$ disk reads!

Insertion — Split on the Way Down

The preferred production approach: split full nodes **proactively** on the way down during insertion, so the actual insert is always into a non-full node (no second pass needed).

Insert Algorithm

```
FUNCTION insert(tree, key):
  r = tree.root
  IF r.n == 2t - 1:    // Root is full
    s = new_node()
    tree.root = s
    s.is_leaf = FALSE
    s.child[1] = r
    split_child(s, 1, r) // Split old root
    insert_non_full(s, key)
  ELSE:
    insert_non_full(r, key)

FUNCTION insert_non_full(node, key):
  i = node.n
  IF node.is_leaf:
    // Shift keys and insert
    WHILE i >= 1 AND key < node.key[i]:
      node.key[i+1] = node.key[i]
      i = i - 1
    node.key[i+1] = key
    node.n += 1
    DISK_WRITE(node)
  ELSE:
    // Find correct child
    WHILE i >= 1 AND key < node.key[i]:
      i = i - 1
    i += 1
    DISK_READ(node.child[i])
    IF node.child[i].n == 2t - 1:
      split_child(node, i, node.child[i])
      IF key > node.key[i]: i += 1
    insert_non_full(node.child[i], key)
```

Split Child Operation

```
FUNCTION split_child(parent, i, full_child):
  // full_child has 2t-1 keys
  new_node = new B-Tree node
  new_node.is_leaf = full_child.is_leaf
  new_node.n = t - 1

  // Copy right half of keys to new_node
  FOR j = 1 TO t-1:
    new_node.key[j] = full_child.key[j+t]

  // Copy right half of children if not leaf
  IF NOT full_child.is_leaf:
    FOR j = 1 TO t:
      new_node.child[j] = full_child.child[j+t]

  full_child.n = t - 1

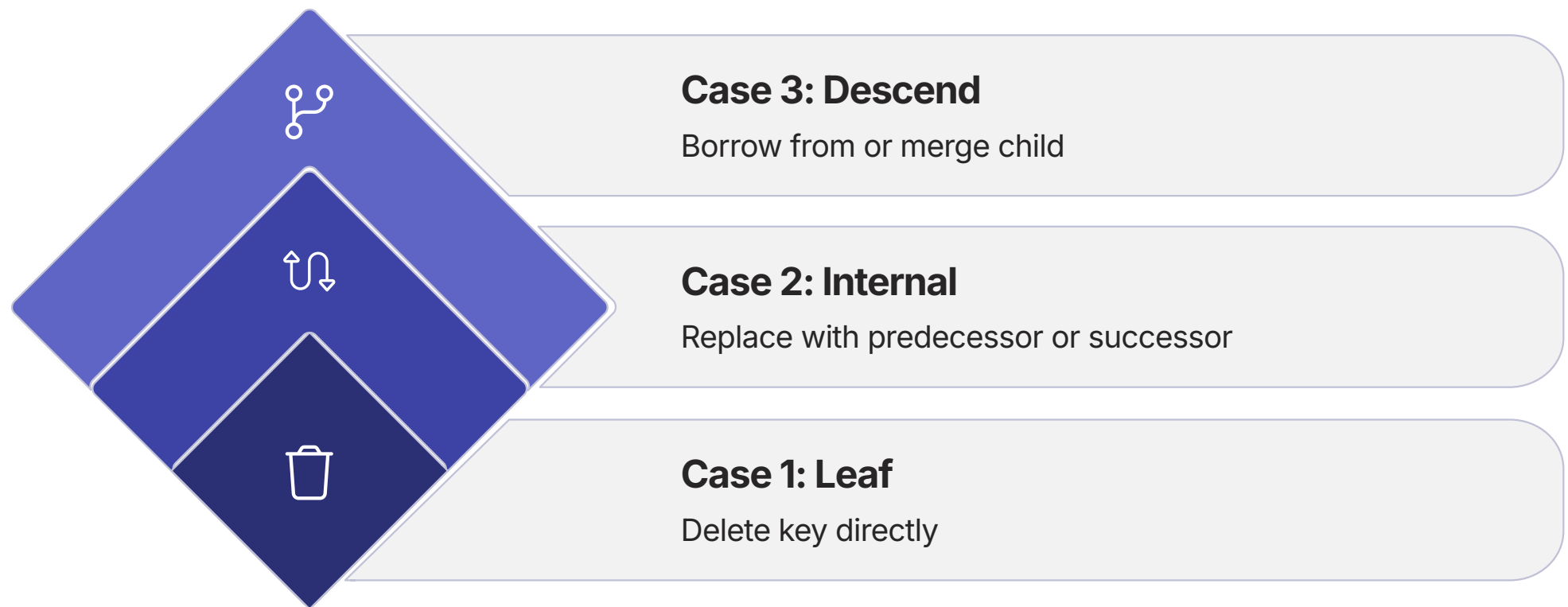
  // Shift parent's children to make room
  FOR j = parent.n+1 DOWNTO i+1:
    parent.child[j+1] = parent.child[j]
  parent.child[i+1] = new_node

  // Push median key up to parent
  parent.key[i] = full_child.key[t]
  parent.n += 1

  DISK_WRITE(full_child)
  DISK_WRITE(new_node)
  DISK_WRITE(parent)
```

✓ Max 1 split per level with proactive splitting — no backtracking needed!

Deletion — Three Main Cases



Deletion is the most complex B-Tree operation, requiring careful handling to maintain the minimum key invariant at every node.

Case 1: Key k is in node x which is a LEAF

→ Simply delete k from x

Case 2: Key k is in node x which is an INTERNAL NODE

2a: Left child (y) has $\geq t$ keys

→ Replace k with predecessor k' (largest in y 's subtree)

→ Recursively delete k' from y

2b: Right child (z) has $\geq t$ keys

→ Replace k with successor k' (smallest in z 's subtree)

→ Recursively delete k' from z

2c: Both y and z have exactly $t-1$ keys

→ Merge $k + z$ into y (y now has $2t-1$ keys)

→ Delete k from merged node

Case 3: Key k is not in node x (descend to child c_i)

3a: c_i has only $t-1$ keys, sibling has $\geq t$ keys

→ Rotate: borrow a key from sibling through parent

3b: c_i and both siblings have $t-1$ keys

→ Merge c_i with a sibling

→ Continue descent

B-Tree Variants

B-Tree (Classic)

Data in all nodes. Full implementation.
General balanced tree use.

B+ Tree

Data only in leaves; leaves linked.
Better range scans. **Database indexes (dominant)**.

B* Tree

Nodes at least 2/3 full (vs. 1/2). Fewer splits; better space utilization. Used in filesystems (HFS+).

Fractal Tree (Bε)

Buffers writes in internal nodes. Write-optimized (TokuDB).
Write-heavy workloads.

Counted B-Tree

Nodes store subtree counts. Enables rank/select. Ordered key-value stores.

B-Tree Internal Structure

- ❑ Internal nodes have data
Pointer to record in node
Range scan: inefficient

```
[10 | data]
 /   \
[5 | data] [20 | data]
```

B+ Tree Internal Structure

- ✅ Internal nodes: only keys (routing)
All data in leaf nodes
Leaves linked → O(k) range scan

```
[10]
 /  \
[5 | d] [10 | d] [20 | d]
←——linked list——→
```

Real-World Use Case: PostgreSQL / MySQL Index

ENTERPRISE DATABASE

Business Problem

An e-commerce platform processes 100,000 orders/day. Queries like `SELECT * FROM orders WHERE customer_id = 42 AND order_date BETWEEN '2024-01-01' AND '2024-12-31'` must execute in milliseconds against a 500GB table.

Solution

PostgreSQL creates a B+ Tree index on `(customer_id, order_date)`. The index fits in a few levels of B+ Tree pages; range queries traverse leaf page linked lists.

✅ **Outcome:** Index scan completes in 2–5ms vs. 30–60 seconds for sequential scan on 500GB.

Query Execution Path

01

Query → Planner → Index Scan

02

B+ Tree Root Page

Usually in buffer pool cache

03

Traverse 2–3 Levels

Usually cached in RAM after warmup

04

Leaf Page Linked List Scan

For range queries

05

Heap Page Fetches

For actual row data

Real-World Use Cases: Filesystems & LMDB



Filesystem Directory Index (NTFS, HFS+, ext4)

Problem: A file server has directories with 100,000+ files. File-open must find a file by name without scanning every entry.

Solution: NTFS uses a B+ Tree to index directory entries by filename. HFS+ uses HFS B-Tree. Linux ext4 uses HTree (a B-Tree variant).

Outcome: File lookup in a 100K-entry directory = $O(\log n)$ disk reads $\approx$ 3–4 reads instead of $O(n)$ sequential reads ($\approx$ 1MB of sequential I/O).



LMDB — Lightning Memory-Mapped Database

Problem: OpenLDAP needed an embedded database with zero-copy reads, ACID compliance, and high performance for key-value workloads.

Solution: LMDB uses a B+ Tree with memory-mapped I/O (mmap). The OS page cache acts as the buffer pool. Copy-on-Write B+ Tree enables MVCC with a single writer and unlimited concurrent readers — zero reader blocking.

Outcome: Read throughput matches RAM speed for hot data; ACID with zero separate WAL needed.

i CoW Architecture: Writer: CoW path copy $\rightarrow$ new version of modified pages $\rightarrow$ atomic root swap | Readers: mmap old pages $\rightarrow$ zero-copy access $\rightarrow$ no locks needed

Implementation: Node Structure & Search

B-Tree Node Structure (Pseudocode)

```
CONSTANT T = 3 // min degree (2–5 keys per node)
```

```
CLASS BTreeNode:
```

```
    n = 0 // current number of keys
```

```
    key[] = array of 2T-1 // sorted keys
```

```
    child[] = array of 2T // child pointers (page IDs)
```

```
    is_leaf = TRUE
```

```
    page_id = 0 // disk page identifier
```

```
CLASS BTree:
```

```
    root = NULL
```

```
    t = T
```

```
FUNCTION new_node():
```

```
    node = new BTreeNode()
```

```
    DISK_WRITE(node) // allocate a disk page
```

```
    RETURN node
```

B-Tree Search Implementation

```
FUNCTION search(node, key):
```

```
    i = 0
```

```
    // Binary search within node
```

```
    // (t can be large → worth it)
```

```
    WHILE i < node.n AND key > node.key[i]:
```

```
        i += 1
```

```
    IF i < node.n AND key == node.key[i]:
```

```
        RETURN (node, i) // Found
```

```
    IF node.is_leaf:
```

```
        RETURN NOT_FOUND
```

```
    DISK_READ(node.child[i]) // One disk I/O
```

```
    RETURN search(node.child[i], key)
```

```
// Disk I/Os = tree height
```

```
// CPU work per node =  $O(\log t)$  with binary search
```

- ❏ Binary search within each node reduces CPU comparisons from $O(t)$ to $O(\log t)$ per level — critical when t is large (e.g., 128 or 256).

Implementation: Full Insertion Walkthrough

Scenario: B-Tree with $t=2$ (max 3 keys, max 4 children). Insert sequence: 1, 2, 3, 4, 5, 6, 7

Insert 1, 2, 3

Root can hold 3 keys: Root: [1, 2, 3]

Insert 5

$5 > 2 \rightarrow$ go right $\rightarrow$ [3,4] not full $\rightarrow$ insert: right child becomes [3,4,5]

Insert 7

$7 > 4 \rightarrow$ go to [5,6] $\rightarrow$ not full $\rightarrow$ insert. Final: root [2,4], leaves [1],[3],[5,6,7]

Insert 4 $\rightarrow$ Split Root

Root is FULL (3 keys = $2t-1$). Median=2 pushed up. Left: [1], Right: [3,4], New root: [2]

Insert 6 $\rightarrow$ Split Child

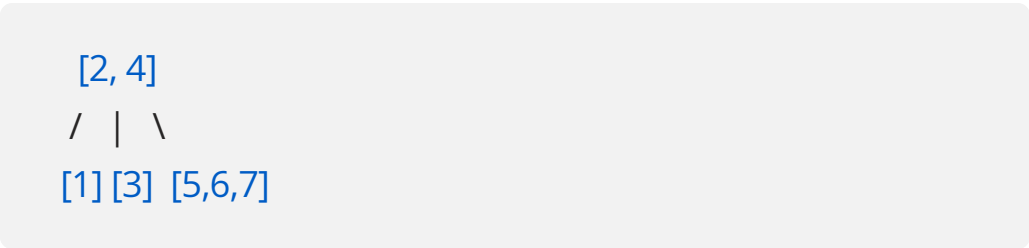
[3,4,5] is FULL $\rightarrow$ split on way down. Median 4 pushed to parent. Parent: [2,4]. Children: [1],[3],[5,6]

✅ **Final Tree:** Root: [2, 4] | Children: [1] | [3] | [5, 6, 7] — All leaves at same depth ✓

Implementation: Delete Walkthrough (Case 3b — Merge)

Scenario: Delete 1

Starting tree (t=2):



t=2 → each non-root node must have $\geq t-1 = 1$ key. Node [1] has 1 key (minimum). Deleting would make it 0 keys → need Case 3.

Step-by-Step

01

Check siblings

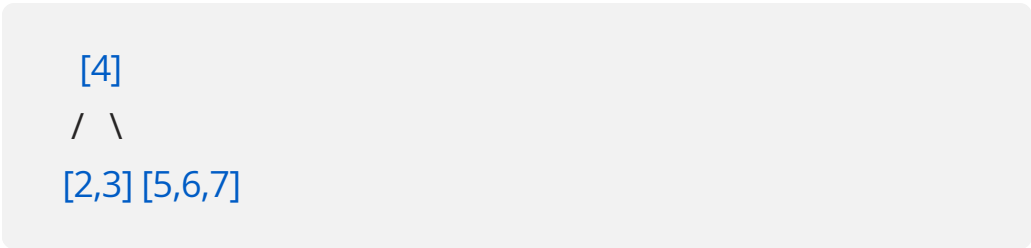
Left sibling of [3] is [1] (t-1 keys), right sibling is [5,6,7] ($\geq t$ keys)

03

Case 3b: Merge

[] is empty → merge [] with [3] using separator 2 → Merged node: [2, 3]

Result



✔ Tree remains valid: all leaves at same depth, all nodes have $\geq t-1$ keys ✓

Key Insight

The merge operation reduces the parent's key count by 1. If the parent was at minimum, this propagates upward — potentially all the way to the root. If the root loses its last key, the tree height decreases by 1.

⚠ Always check sibling key counts before deciding merge vs. borrow (Case 3a vs. 3b). Unnecessary merges cause the tree to shrink faster than needed.

02

Case 1 applies

[1] is a leaf → delete 1 directly → [1] becomes []

04

Update parent

Parent loses key 2 and child pointer to []

Hands-On Lab 1: B-Tree Construction Simulation

BEGINNER-INTERMEDIATE

Objective

Simulate a B-Tree of degree $t=2$ on paper and in code; understand split behavior.

Implementation Steps

1 Define BTreeNode

With: `keys[]`, `children[]`, `is_leaf`, `n` (key count)

2 Implement `search(node, key)`

Returns `(node, index)` or `NOT_FOUND`

3 Implement `split_child(parent, i, child)`

Splits a full child node

4 Implement `insert(tree, key)`

Using the split-on-way-down approach

5 Insert sequence

`[3, 7, 1, 5, 11, 9, 2, 13, 4, 8]` with $t=2$; print tree level-by-level after each insert

6 Implement `verify_btree(root, t)`

Check all invariants: same leaf depth, min keys, sorted keys, ordering property

Expected Final Tree ($t=2$)

Level 0: `[5, 9]`

Level 1: `[1,2,3,4]` `[7,8]` `[11,13]`

Exact structure may vary by split ordering — verify with property checker

Troubleshooting Tips

Verification Checklist

- All leaves at same level
- Every non-root internal node has $\geq t$ children
- Every node has $\leq 2t-1$ keys
- Keys within each node are sorted
- All child keys satisfy B-Tree ordering property

Median Index

Median is index `t-1` (0-indexed) of a full $2t-1$ key array

Parent Key Count

Remember to update parent's key count (`n`) after splitting

Hands-On Lab 2: Disk-Aware B-Tree with Page Simulation

ADVANCED

Objective

Simulate disk page management, buffer pool, and measure disk I/Os for different tree configurations.

Implementation Steps

Expected Insight

As t increases:

- fewer disk I/Os per operation (shorter tree)
- BUT **each** node **read** brings more data
- wasted **if** most **keys not** needed

Optimal t : **when** node size $\approx$ disk page size
(typically 4KB–16KB)

✔ **Validation:** For $t=100$, $n=10,000$: search should need ≤ 3 disk reads.

Learning Outcome

Deeply understand **WHY B-Trees exist** and how to tune t for a given storage system. This lab bridges theory and production engineering — the same reasoning used by PostgreSQL and MySQL engineers when tuning page sizes.

ℹ **Prerequisites:** File I/O, understanding of buffer pools, complete Lab 1

01

Disk Simulator

A file (or large array) indexed by page ID. `disk_read(page_id)` and `disk_write(page_id, node)` increment I/O counter.

02

Buffer Pool

Cache the last N pages with LRU eviction. All node accesses go through `disk_read/write`.

03

Insert 10,000 random keys

Measure total disk I/Os

04

Run 1,000 random searches

Measure total disk I/Os

05

Vary t (2, 5, 10, 50, 100)

Measure how I/O count changes; plot t vs. average disk I/Os per search

Advantages & Disadvantages

✓ Advantages



Disk I/O Efficiency

Minimizes disk accesses
— $O(\log_t n)$, near-constant for large t



Always Balanced

All leaves at same depth
— guaranteed worst-case performance



Range Queries

In-order traversal is sequential disk access — efficient



Self-Organizing

Automatically maintains balance on insert/delete



Scalable

Works from kilobytes to terabytes — same algorithm



Proven

50+ years of production use; extremely well understood

✗ Disadvantages

In-Memory Overhead

Large nodes waste memory if most keys are unused

Write Amplification

One logical write can cause many page writes (splits/merges + WAL)

Implementation Complexity

Delete with merge/rotate across 6+ cases is notoriously hard to get right

Point Lookup vs. Hash

For pure point lookups, a hash index is faster ($O(1)$ vs. $O(\log n)$)

Write-Heavy Workloads

LSM Trees often outperform B-Trees for very write-heavy workloads

Space Utilization

Nodes may be only 50–70% full after many deletes (without redistribution)

Performance Considerations

Complexity Analysis

Operation	Disk I/Os	CPU Work
Search	$O(\log_t n)$	$O(t \times \log_t n)$
Insert	$O(\log_t n)$	$O(t \times \log_t n)$
Delete	$O(\log_t n)$	$O(t \times \log_t n)$
Space	$O(n/t)$ nodes	—

Practical Numbers (t=100, n=10⁹)

Height = $\log_{100}(10^9) = 4.5 \rightarrow 5$ levels

Disk I/Os **per** lookup: 5

At 10ms **per** disk I/O: 50ms **per** lookup

At 100µs **per** SSD I/O: 0.5ms **per** lookup

With buffer pool (root + level 1 cached):

3 I/Os **from** warm cache $\rightarrow$ 0.3ms SSD

Key Metrics to Monitor in Production

Optimization Techniques

- **Bulk Loading:** Load sorted data bottom-up $\rightarrow O(n/t)$ disk writes vs. $O(n \log_t n)$ for sequential inserts
- **Page Prefetching:** For range scans, prefetch next leaf page while processing current
- **Compression:** Compress node contents to pack more keys per page
- **Buffer Pool Tuning:** Keep root and upper levels always in buffer pool (hot pages)

1

Fill Factor
Average % of node capacity used. Target: 70–90%

2

Tree Height
Alert if height exceeds expected $\log_t(n)$

3

Split Rate
High splits = heavy insert load; may need higher t

4

Merge Rate
High merges = heavy delete load; may need redistribution

Scalability & Reliability

Scaling Behavior by Degree t

Data Size	t=10 Height	t=10 Height	t=50 Height
1,000	3	2	2
1,000,000	6	4	3
1,000,000,000	9	5	4
1,000,000,000,000	12	6	5

Shadow Paging (LMDB approach)

Copy-on-Write: modified pages written to new disk locations

New root page written last (atomic pointer swap)

Old pages reclaimed after all readers release them

→ No WAL needed; no torn writes

Write-Ahead Logging (WAL)



If crash occurs → replay WAL to restore B-Tree state. This is how PostgreSQL, MySQL, and SQLite ensure durability.

Replication Pattern

01	02	03
Primary writes B-Tree	Replicas replay WAL	Failover
WAL shipped to replicas	Maintain identical B-Tree state	Promote replica — complete tree state available

Design & Architectural Considerations

Key Design Decisions

Decision	Options	Trade-off
Minimum Degree t	Smaller t (2–5) or larger t (128–512)	Larger t = fewer I/Os but more CPU per node
Node size	Match disk page (4KB, 8KB, 16KB)	Mismatch wastes I/O bandwidth
Data in internal nodes	B-Tree (yes) vs. B+ Tree (no)	B+ Tree better for range scans; B-Tree better for point lookups
Buffer pool	LRU, Clock, 2Q, ARC	Choose based on access patterns; hot pages pinned
Concurrency	Page-level locks vs. B-link tree	B-link tree supports concurrent insert without full tree lock
Write strategy	In-place update vs. Copy-on-Write	CoW enables MVCC; in-place needs WAL

B-Link Tree (Production Concurrency)

The standard way to implement a concurrent B-Tree:



Each node has a "high key" (rightmost key in node) and a "link pointer" (→ right sibling). Allows concurrent readers to follow a split in progress. No global lock needed; only page-level locks. **Used in: PostgreSQL's B-Tree index implementation.**

Architecture Patterns

Database Index Layer

B+ Tree over heap file → most relational databases

Covering Index

Include all query columns in leaf nodes → avoid heap fetch entirely

Clustered Index

B+ Tree where leaf nodes ARE the data rows → InnoDB's primary key index

Composite Index

Multi-column key → prefix queries use the index; non-prefix queries don't

Security Considerations

Page Corruption

Impact: Corrupted B-Tree pages → wrong query results or crashes.

Mitigation: Page checksums (PostgreSQL: `page_verify_checksums`); detect corruption on read.

Write Amplification Attack

Impact: Adversary triggers maximum splits/merges to exhaust I/O.

Mitigation: Rate limiting; WAL write throttle.

Index Bloat

Impact: Repeated insert+delete leaves partially-filled pages → wasted space.

Mitigation: Periodic `VACUUM`/`REINDEX`; fill factor tuning.

Stale Read

Impact: Concurrent read sees partial write (without MVCC).

Mitigation: MVCC with snapshot isolation; WAL replay validation.

Unauthorized Index Access

Impact: Index exposes data ordering/key values.

Mitigation: Column-level access control; encrypted storage engine.

Best Practices

✓ Do's

- **Size nodes to match storage block/page size**
Calculate t from disk page size and key+pointer sizes
- **Use B+ Trees for range queries**
Linked leaf pages make range scans dramatically faster
- **Implement `verify_btree()` in all tests**
Run property checker to catch subtle bugs early
- **Bulk-load sorted data bottom-up**
Dramatically faster than sequential inserts — $O(n/t)$ vs. $O(n \log_t n)$ disk writes
- **Pin hot pages in buffer pool**
Keep root and level-1 pages permanently cached
- **Always use Write-Ahead Logging (WAL)**
Essential for crash recovery and durability

✗ Don'ts

Don't use B-Trees for in-memory only workloads

AVL/Red-Black Trees are better suited for pure in-memory use

Don't use fill factor of 100%

Leaves no room for inserts without immediate splits

Don't ignore the delete merge case

Incomplete delete causes nodes to become empty — tree corruption

Don't scan a B+ Tree randomly

Sequential leaf traversal is efficient; random jumps kill I/O performance

Don't skip vacuuming

Deleted keys leave "tombstone" space; trees bloat over time

Common Mistakes & Pitfalls

Beginner Mistakes

Mistake	Why	Impact	Fix
Confusing t (min degree) with order (max children)	Terminology varies by textbook	Wrong node capacity calculations	order $m = 2t$; each node has $t-1$ to $2t-1$ keys
Not splitting root before insert	Root fill check forgotten	Root exceeds max keys $\rightarrow$ tree structure broken	Always check and split root first if full
Treating B-Tree like a BST	B-Trees are multi-way	Incorrect child indexing	k keys $\rightarrow k+1$ children

Intermediate Mistakes

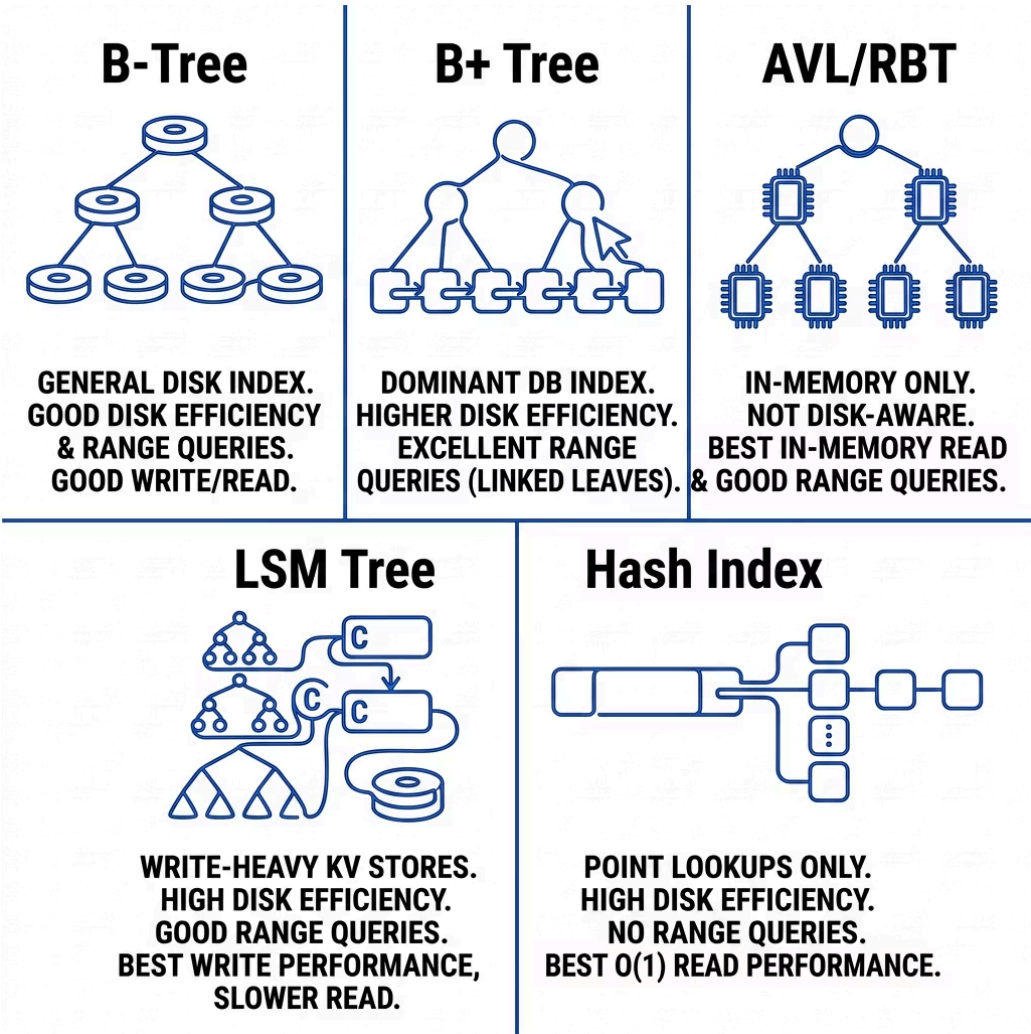
Mistake	Why	Impact	Fix
Not redistributing before merging in delete	Case 3a (borrow from sibling) skipped	Unnecessary merges cause tree to shrink faster than needed	Always check sibling key counts before deciding merge vs. borrow
Node size doesn't match disk page	Arbitrary node size chosen	Extra I/Os: a node spans multiple pages or wastes a page	Calculate t from disk page size and key+pointer sizes

Advanced / Production Mistakes

Mistake	Why	Impact	Fix
No WAL on write path	Assumed disk writes are atomic	Data loss / corruption on crash	Implement WAL; write log before modifying tree pages
Hot root page not cached	Buffer pool evicts all pages equally	Root re-read from disk on every lookup	Pin root and level-1 pages permanently in buffer pool
No page checksums	Bit rot or OS bugs silently corrupt pages	Silent wrong query results	Enable page checksum verification on every read
Ignoring write amplification	Each logical write triggers multiple page writes	SSD wear; throughput degradation under heavy write load	Monitor write amplification factor; tune fill factor; consider LSM for write-heavy workloads

Comparison with Alternatives

Understanding when to choose B-Trees versus alternatives is critical for system design. Each structure excels in different scenarios based on access patterns, data size, and consistency requirements.



Choose B+ Tree when

Building a relational database index with range queries (PostgreSQL, MySQL)

Choose AVL/RBT when

In-memory only, no disk persistence needed, best read performance

Choose LSM Tree when

Write-heavy workloads dominate (LevelDB, RocksDB, Cassandra)

Choose Hash Index when

Pure point lookups, no range queries needed (Redis, Memcached)

Feature	B-Tree	B+ Tree	AVL/RBT	LSM Tree	Hash Index
Disk Efficiency	✔ High	✔ Higher	✗ No	✔ High	✔ High
Range Queries	✔ Good	✔ Excellent	✔ Good	✔ Good	✗ None
Write Performance	✔ Good	✔ Good	✔ Good	✔✔ Best	✔ Good
Read Performance	✔ Good	✔ Good	✔ Best	⚠ Slower	✔ Best O(1)
ACID / Transactions	✔ WAL	✔ WAL	✔	Partial	Limited
Examples	Early DBs	PostgreSQL, MySQL	In-memory caches	LevelDB, RocksDB	Redis, Memcached

Learning Summary & Quick Reference

Key Takeaways

- 1

Minimize Disk I/O

One node = one disk page — the defining design goal
- 2

$O(\log_t n)$ Height

With $t=128$, a billion-record tree needs only ~4 disk reads per lookup
- 3

Node Invariants

 $t-1$ to $2t-1$ keys; all leaves at the same depth
- 4

Insert: Split-on-Way-Down

Proactive splitting ensures the actual insert is always into a non-full node
- 5

Delete: Three Cases

Leaf deletion, internal node deletion, propagation with merge/borrow
- 6

B+ Trees Dominate Production

All data in leaves, leaves linked — powers every major RDBMS index

B-Tree Quick Reference

📄

MIN DEGREE t :
Keys per node: $t-1$ to $2t-1$
Children per node: t to $2t$
Root special: 1 to $2t-1$ keys
All leaves at same depth ✓

SEARCH: Traverse root→leaf; 1 disk read per level

INSERT:
1. Split full nodes on the way down
2. Insert into guaranteed non-full leaf
Max 1 split per level

DELETE (3 cases):
Leaf → direct delete
Internal → predecessor/successor replacement
Propagate → borrow from sibling or merge

COMPLEXITIES:
All operations: $O(\log_t n)$ disk I/Os
Height: $\leq \log_t((n+1)/2)$
Optimal t : $\text{node_size} = \text{disk_page_size}$

Memory Aids

“

"B = Broad" — wide nodes, short height, fewer disk trips

”

“

"All leaves on the same floor" — the defining balance property

”

“

"Split before you're full" — proactive splitting prevents backtracking

”

Interview Questions & Next Steps

Common Interview / Certification Questions

01

Why B-Trees instead of AVL/Red-Black for DB indexes?

02

What is minimum degree t , and what does it control?

03

How does a B-Tree ensure all leaves are at the same depth?

04

Explain the split operation — what happens when a node is full during insert?

05

What is the difference between a B-Tree and a B+ Tree?

06

How does deletion handle the case where a node would become underfull?

07

If disk page = 8KB, key = 8 bytes, pointer = 8 bytes, what is optimal t ?

08

How do databases recover from a crash when using B-Trees?

09

What is write amplification, and when is it a problem?

10

When would you choose an LSM Tree over a B-Tree?

Recommended Next Topics



B+ Trees

The production-dominant variant with linked leaf pages — the next logical step



Write-Ahead Logging (WAL)

Crash recovery for B-Tree based systems — essential for production



Buffer Pool Management

LRU, Clock, ARC eviction policies — critical for performance tuning



LSM Trees

Log-Structured Merge Trees for write-heavy workloads — the main alternative



Database Indexing

Clustered vs. non-clustered, covering indexes, composite indexes